

Abstract	2
Introduction	2
Background study	3
Existing literature review	3
Pre-trained VS Fine-tuned model:	4
ANPR using Haar cascade classifier	5
ANPR using histogram of gradients	6
Methodology	7
General overview	7
Pipeline walkthrough	9
Data preprocessing	11
YOLOv8 model architecture	14
High-level architecture	15
1. Input Processing:	15
2. Backbone for Feature Extraction:	15
3. Feature Fusion Network (Neck):	15
4. Generation of Feature Maps (P3, P4, P5):	15
5. Prediction Head:	15
6. Output Processing:	1
Expanded architecture	1
1. Backbone: Feature Extraction with CSPDarknet53	1
2. Neck: Feature Fusion with Path Aggregation Network (PAN)	1
3. Head: Detection with YOLO Layer	1
Deep sort model architecture	1
OCR using EasyOCR	1
Typical Workflow with EasyOCR:	1
Use Cases of EasyOCR:	1
Graphical user interface	1
Results and evaluations	1
Challenges and limitations	1
Conclusion	1
References	1
Appendix	1

Abstract

This project explores and evaluates various methodologies for automatic number plate recognition (ANPR) and proposes a novel approach that integrates different state-of-the-art techniques. The resulting model achieves optimal performance and can be seamlessly integrated into hardware devices for real-time number plate detection and authorization. By combining advanced algorithms and practical considerations, the proposed ANPR system demonstrates high accuracy, efficiency, and suitability for deployment in diverse real-world applications. This research contributes to the ongoing advancement of ANPR technology, offering promising prospects for enhanced efficiency and reliability in automated number plate recognition systems.

Keywords: Automatic number plate recognition (ANPR), YOLOv8, Object detection, Optical character recognition, Deep learning

Introduction

In our fast-paced world, where technology continually evolves, one notable area of advancement is in automated surveillance and traffic management systems. Among these systems, automatic number plate detection and recognition (ANPR) holds particular significance due to its potential to streamline various processes and enhance security measures in urban environments. These systems are designed to automatically identify and recognize licence plates from images or video streams captured by surveillance cameras. These systems find applications in law enforcement, traffic management, parking management, and electronic toll collection, offering benefits such as improved efficiency and enhanced security. Despite their potential, ANPR systems face challenges, including the need for accurate detection and recognition algorithms capable of handling diverse environmental conditions. Additionally, ensuring privacy and data security remains a critical concern.

This project aims to tackle these challenges by leveraging computer vision, deep learning, and optical character recognition (OCR) technologies to create an efficient ANPR system. By combining advanced object detection algorithms with OCR, the system can autonomously detect vehicles, locate licence plates, and extract alphanumeric characters. Moreover, incorporating vehicle tracking capabilities enhances system reliability.

Through this project, we aim to contribute to advancing automated surveillance and traffic management systems, offering a solution that improves efficiency and security in urban

environments. Furthermore, we emphasise the importance of addressing ethical and privacy considerations in developing and deploying ANPR systems.

In this report, we delve into the technical details of our approach, highlighting methodologies, algorithms, and implementation strategies. The first section of the report explores and reviews existing work in automatic number plate recognition, followed by a methodology section, where the system architecture, tools and technologies, algorithms, models, and libraries are explained in detail. The results section discusses the overall reliability and accuracy of the proposed system, highlighting potential areas for improvement and future work. The conclusion briefly summarises the entire project report and concludes the discussion.

Background study

Existing literature review

There has been a lot of research done in automated number plate detection and recognition using a wide variety of machine learning and deep learning algorithms. The existing ANPR methods have been effective for various image categories but face challenges with low lighting or blurred images. To address these challenges, research has focused on developing improved algorithms using deep learning and computer vision techniques (Li et al., 2019; Luo & Liu, 2022; Xie et al., 2018). For instance, a novel approach based on YOLOv5m and LPRNet has been proposed to enhance recognition accuracy and real-time performance in complex scenes (Luo & Liu, 2022). Additionally, the use of ANPR data has been instrumental in predicting traffic flow and creating anonymized linked open datasets on urban bustles (Vyvere & Colpaert, 2022). ANPR systems have also been integrated into smart parking systems, contributing to efficient parking management (Narendra et al., 2023).

Furthermore, the integration of ANPR data with deep learning algorithms has shown promise in improving the efficiency and accuracy of licence plate location (Zhu et al., 2022). The potential of ANPR systems in crime prediction and prevention has also been explored, highlighting the role of machine learning and computer vision in forecasting criminal activities (Shah & Shah, 2021). Moreover, the availability of massive traffic data from ANPR systems has enabled a better understanding of day-to-day traffic variability in urban networks (Barroso et al., 2021; Sánchez-Cambronero et al., 2021).

The application of ANPR extends to various domains, including toll automation, traffic flow analysis, and vehicle tracking (Sánchez-Cambronero et al., 2021; Barroso et al., 2021; Mane et al., 2019). Moreover, research has emphasised the importance of efficient methods for licence plate localization and recognition, utilising statistical features and neural networks (Rehman et al., 2020). Additionally, the integration of ANPR with artificial intelligence and optical character recognition has been explored for vehicle identification and traffic management (Alizadeh & Luqman, 2021).

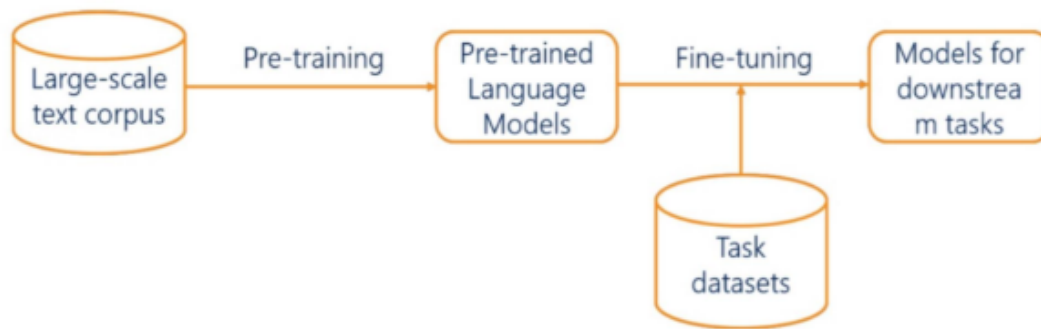
Pre-trained VS Fine-tuned model:

We used a pre-trained YOLOv8 model for detecting the cars in videos. Pre-trained models are neural network architectures that have been trained on a large dataset for a specific task, typically using a vast amount of labelled data. The advantages of pre-trained models are:

1. **Feature Extraction:** Pre-trained models encode valuable information learned from a large dataset, which can be leveraged as feature extractors for related tasks. This allows researchers and practitioners to benefit from the knowledge gained during pre-training without the need to train a model from scratch.
2. **Transfer Learning:** Pre-trained models facilitate transfer learning, wherein the knowledge learned from a source task (pre-training) is transferred to a related target task. By fine-tuning the pre-trained model on a smaller dataset specific to the target task, one can achieve faster convergence and often better performance compared to training from scratch.
3. **Generalisation:** Pre-trained models tend to generalise well to unseen data, thanks to the diverse set of patterns they have learned during pre-training. This generalisation capability is particularly advantageous when working with limited amounts of task-specific data.

Fine-tuning involves taking a pre-trained model and further training it on a specific dataset or task of interest. During fine-tuning, the parameters of the pre-trained model are adjusted to better fit the nuances of the target task, typically using a smaller dataset that is specific to the target domain. For our use case, we fine-tuned the YOLOv8 model on a licence plate dataset for detecting the licence plates of cars in videos. The advantages of fine-tuned models are

1. **Task-Specific Adaptation:** Fine-tuning allows the model to adapt to the intricacies of the target task or dataset by learning task-specific patterns and features. This process enables the model to achieve higher performance on the target task compared to using the pre-trained model alone.
2. **Domain Adaptation:** Fine-tuning can help mitigate domain shift, where the distribution of data in the target domain differs from that of the source domain used for pre-training. By fine-tuning on target domain data, the model can better align its representations with the target distribution, improving performance on the target task.
3. **Customization:** Fine-tuning offers flexibility in model customization, allowing practitioners to tailor the pre-trained model to suit their specific application requirements and domain characteristics.



The figure above shows the process of pre-training and fine-tuning. Pre-training a model includes taking a large-scale dataset and supplying it to the model, the result is a pre-trained model. On the other hand, fine-tuning involves taking a task-specific smaller dataset and training the pre-trained model on that dataset, resulting in a fine-tuned model specialising in a particular task such as licence plate detection in our case.

ANPR using Haar cascade classifier

Haar cascade classifier is a machine learning-based approach used for object detection. It operates by analysing the intensity gradients in an image using a set of trained classifiers. These classifiers are organised in a cascade structure to efficiently filter out regions of the image that are unlikely to contain the object of interest. The training process involves providing positive and negative samples to train the classifier. Positive samples are images containing the object of interest (in this case, number plates), while negative samples are images without the object. During training, the classifier learns to distinguish between positive and negative samples based on features extracted using Haar-like features.

To detect licence plates using the OpenCV Haar cascade classifier, we first train a classifier using positive images (images of number plates) and negative images (images without number plates). The classifier is trained to recognize the features of a number plate, such as its shape, size, and texture. Once the classifier is trained, it can be used to detect licence plates in new images by applying the classifier to the image and looking for areas that match the features of a number plate

While the Haar classifier is easy to set, and use, and a fast method for detecting licence plates, it is not accurate and prone to false positives. Moreover, it also requires a huge number of training images, as compared to other deep learning-based methods. Lastly, images with complex backgrounds, low lighting conditions, and blurry images tend to mislead the classifier to produce incorrect results. Therefore, it is not suitable for systems where high security is required, as the classifier might produce incorrect outputs and lead to security breaches.

ANPR using histogram of gradients

The Histogram of Gradients (HOG) is a feature descriptor technique used for object detection in images. It operates by analysing the distribution of gradient orientations in localised regions of an image. The process involves the following steps:

1. Gradient computation: The first step in HOG is to compute the gradients of images in both the x and y directions. Let I be the input image, D_x be our filter in the x-direction, and D_y be our filter in the y-direction, then the equation to compute gradients and magnitude is:

$$G_x = I \star D_x \text{ and } G_y = I \star D_y$$

$$|G| = \sqrt{G_x^2 + G_y^2}$$

We then calculate the angle of the gradient for each pixel in the original image using the equation below:

$$\theta = \arctan2(G_y, G_x)$$

Given both $|G|$ and θ , we can now compute a histogram of oriented gradients, where the bin of the histogram is based on θ and the contribution or weight added to a given bin of the histogram is based on $|G|$. The gradient magnitude denotes the intensity change at each pixel, while the gradient orientation indicates the direction of this change.

2. After this, we divide the image into cells of equal number of pixels and then calculate a histogram of gradients for each cell. To create a histogram for a single cell, the following steps are used:
 - Compute the gradient magnitude and orientation for each pixel within the cell. This process is typically performed using gradient operators such as the Sobel operator.
 - Define a set of orientation bins covering the range of possible gradient orientations. These bins are evenly spaced intervals, usually ranging from 0 to 180 degrees.
 - Assign each pixel within the cell to its corresponding orientation bin based on its gradient orientation. Accumulate the gradient magnitudes of all pixels within the cell into the appropriate bins. This accumulation entails adding the gradient magnitude of each pixel to the bin corresponding to its orientation.
3. After the histogram is computed, it may be normalised to ensure that the descriptor is invariant to changes in illumination and contrast. Normalisation techniques, such as L2 normalisation, are often applied to each histogram within a larger region called a block.

Based on the same general technique, (Omar et al., 2019) utilized the Histogram of Gradients method on a vehicle dataset for automatic number plate detection. The researchers devised a method to detect vehicle number plates within images through a combination of image processing methods and detection techniques. It encompassed a training phase, wherein positive and negative samples are utilized, and a subsequent testing phase to evaluate the trained model's performance. The Cascade Detector Trainer component employs the AdaBoost technique to train a classifier capable of accurately identifying regions of interest (ROIs) containing number plates. To precisely localize ROIs in vehicle images, the system utilizes a Histogram of Oriented Gradients (HOG) descriptors. Experimental results indicate that the developed model achieved an accuracy score of 89.66% on the dataset, demonstrating its efficacy in accurately detecting vehicle number plates.

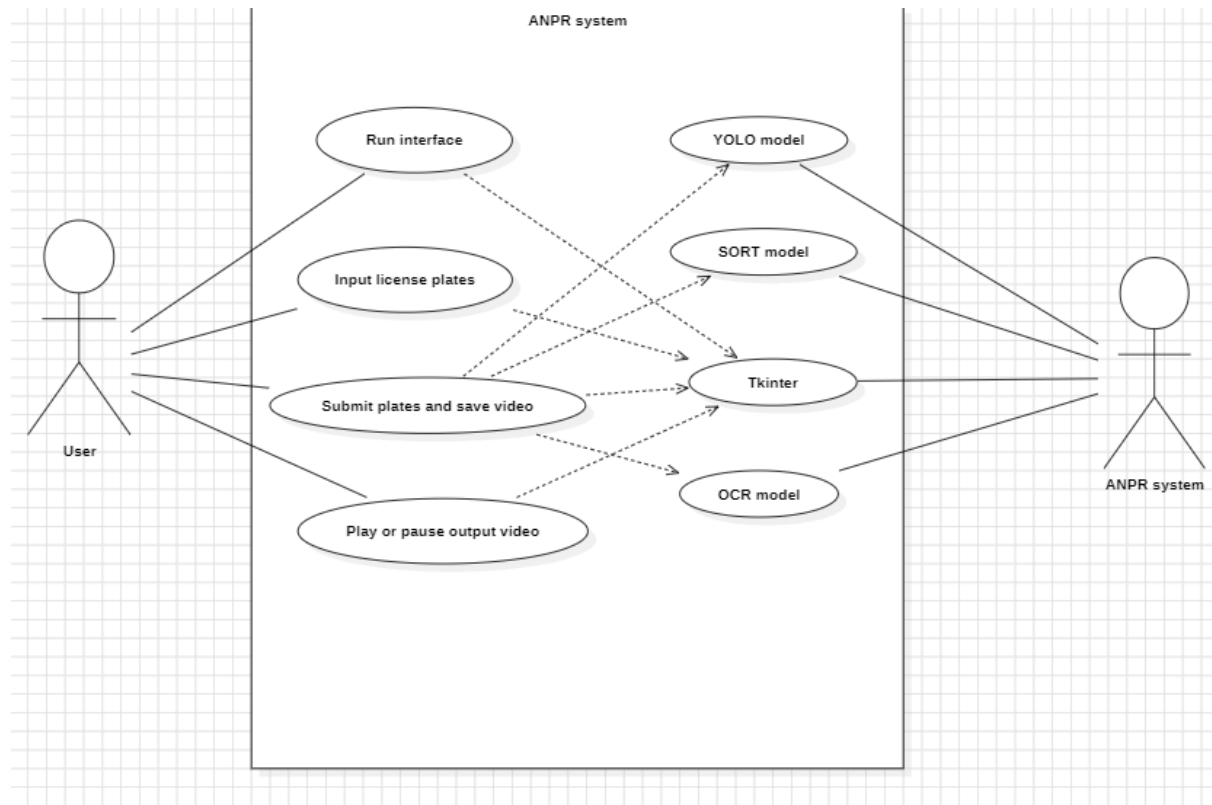
Methodology

Our methodology involved a wide variety of techniques and models, they were integrated to create a unified pipeline.

General overview

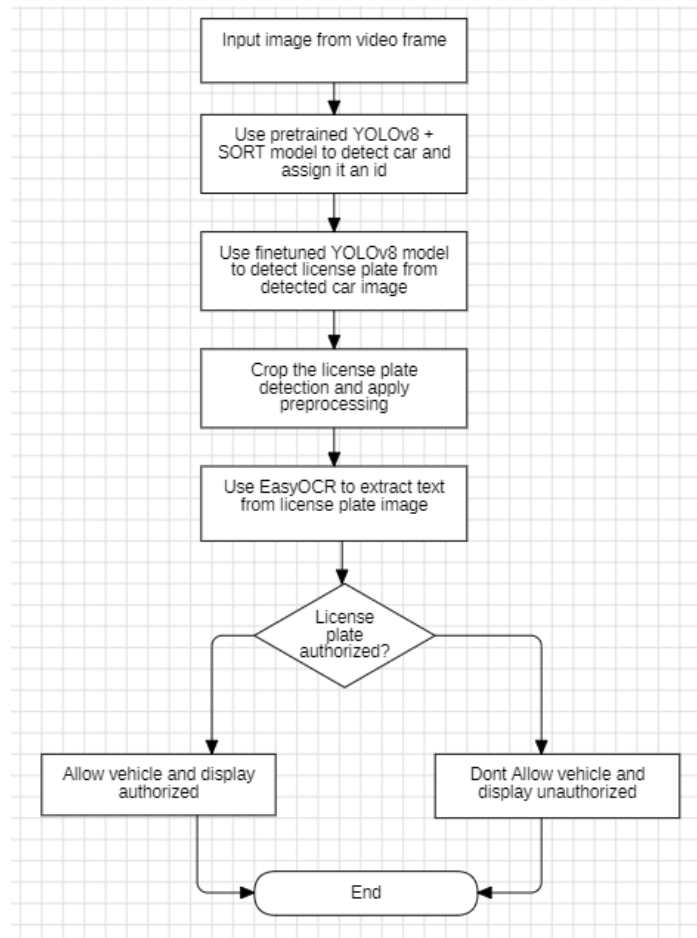
A general overview of every model/algorithm is described below, which we will then expand on in detail:

- Pre-trained YOLO model: Utilised to identify and localise cars within images or video frames. YOLO, or You Only Look Once, is a deep learning-based object detection algorithm that efficiently detects objects in real-time, making it suitable for the rapid processing of car detection tasks.
- YOLOv8 fine-tuned model: Specifically fine-tuned to excel in recognizing licence plates within the detected cars. YOLOv8 is a variant of the YOLO architecture that has been optimised and fine-tuned to achieve high accuracy in identifying licence plates, leveraging training data and techniques tailored to licence plate recognition tasks.
- DEEP SORT: Employed for the multi-object tracking of cars across consecutive frames. DEEP SORT integrates deep learning techniques with traditional tracking algorithms to associate and track multiple cars over time, ensuring consistent and reliable tracking even in challenging scenarios such as occlusions and varying viewpoints.
- EasyOCR: Implemented for extracting text from the identified licence plates. EasyOCR is a Python library that provides an easy-to-use interface for optical character recognition (OCR), specifically designed for reading text from images. It enables efficient extraction of alphanumeric characters from licence plates, facilitating downstream tasks such as licence plate recognition and analysis.



The above use case diagram shows how the user will interact with the system. The user has the option to run the interface, input licence plates, submit plates and save video, and then play or pause the generated video. The user inputs the licence plates via the interface, the list of plates is sent to the model which then highlights all unauthorised plates in RED and the remaining authorised ones in GREEN. Once the video is generated, it is shown in the interface and the user can play or pause the video easily.

- The input to the system is a live video feed from the surveillance cameras installed at the intersection.
2. Object Detection with Pre-trained YOLO Model:
 - The pre-trained YOLO model analyses each frame of the video feed and detects cars present in the scene. For each detected car, the model provides bounding box coordinates.
 3. Fine-tuned YOLOv8 Model for licence Plate Detection:
 - The fine-tuned YOLOv8 model, specialised in licence plate detection, focuses on the bounding boxes provided by the pre-trained YOLO model. It accurately identifies and localises licence plates within the detected cars.
 4. DEEP SORT for Multi-object Tracking:
 - The DEEP SORT algorithm tracks each car detected by the pre-trained YOLO model and associates it with its corresponding licence plate detected by the fine-tuned YOLOv8 model. This ensures consistent tracking of vehicles across consecutive frames, maintaining unique identities for each car.
 5. EasyOCR for licence Plate Text Extraction:
 - Once the licence plates are detected and associated with their respective cars, EasyOCR is employed to extract the text from each licence plate. This step provides the alphanumeric characters present on each licence plate.
 6. Output:
 - The output of the pipeline is extracted licence plate text for every detected car.



The figure above shows how the process will work for a single image input to the automatic number plate recognition system

Data preprocessing

The dataset utilised for this project was sourced from Kaggle and is titled "licence PLATE DATASET" (available at: <https://www.kaggle.com/datasets/chiraggodaw/licence-plate-dataset-resized>). It comprised a total of 12,000 images capturing various licence plates. To facilitate model training and evaluation, a subset of 3,000 images was carefully selected. This subset was then split into three subsets according to a 70:20:10 ratio for training, validation, and testing, respectively. By adhering to this ratio, 70% (2,100 images) of the dataset was allocated for training the models, 20% (600 images) for validating the model's performance during training, and the remaining 10% (300 images) for assessing the model's generalisation ability on unseen data. This approach ensured a balanced distribution of data across the training, validation, and testing sets, enabling effective model development and evaluation while minimising the risk of overfitting.

Following image augmentation and preprocessing steps applied to the "licence PLATE DATASET" for training the models:

1. Image Resizing:

- All images in the dataset were resized to a standardised size (e.g., 416 x 416 pixels) to ensure consistency in input dimensions across the dataset. Resizing helps in reducing computational complexity and improving model efficiency.

2. Brightness Adjustment:

- Brightness adjustment involves altering the intensity of pixel values uniformly across the entire image. This augmentation technique simulates variations in lighting conditions that can occur in real-world scenarios. By randomly adjusting the brightness of images in the dataset, we introduce variability that helps the model learn to recognize licence plates under different lighting conditions.
- Implementation: To adjust brightness, each pixel value in the image is modified by adding or subtracting a random value within a specified range. For example, to increase brightness, a positive value is added to each pixel; to decrease brightness, a negative value is added. The range of values added/subtracted can be determined based on the desired level of augmentation and the dynamic range of pixel values in the images.

3. Contrast Adjustment:

- Contrast adjustment involves changing the difference in intensity between the brightest and darkest parts of the image. Similar to brightness adjustment, contrast augmentation is essential for making the model invariant to changes in contrast levels that can occur in real-world scenarios. By randomly adjusting the contrast of images in the dataset, we expose the model to a broader range of contrast variations, improving its ability to generalise.
- Implementation: Contrast adjustment can be implemented by applying a linear transformation to the pixel values of the image. This transformation scales the pixel values to increase or decrease the contrast. For instance, to increase contrast, pixel values are stretched across a wider range; to decrease contrast, pixel values are compressed into a narrower range. The amount of contrast adjustment can be controlled by adjusting the parameters of the linear transformation, such as the contrast multiplier.

4. Noise Addition:

- Noise addition involves introducing random variations into the pixel values of the image, simulating the noise commonly present in real-world images captured by cameras or sensors. Common types of noise added include Gaussian noise, which follows a Gaussian distribution, and random noise, which is uniformly distributed. Noise addition is beneficial for training models to be robust to noisy environments and enhances their generalisation ability.
- Implementation: Gaussian noise can be added by generating random values from a Gaussian distribution with zero mean and a specified standard deviation, and adding these values to the pixel values of the image. Similarly, random noise can be added by generating random values within a certain

range and adding them to the pixel values. The amount of noise added can be controlled by adjusting the standard deviation or range of the random values.

5. Blur Effect:

- The blur effect involves smoothing out the high-frequency components in the image by applying a blurring filter. Gaussian blurring and median blurring are common techniques used to blur images with varying kernel sizes. Blurring augmentation helps in reducing overfitting by reducing the details in the image and enhancing the model's ability to generalise to unseen data.
- Implementation: Gaussian blurring applies a convolution operation with a Gaussian kernel to the image, resulting in a weighted average of neighbouring pixel values. The size of the Gaussian kernel determines the extent of blurring, with larger kernels resulting in more smoothing. Median blurring replaces each pixel value with the median value of its neighbouring pixels, effectively reducing the impact of outliers and noise in the image. The size of the median filter determines the degree of blurring.

6. Thresholding:

- Thresholding is a technique used to convert grayscale images to binary images by dividing the pixel values into two groups based on a specified threshold value. In licence plate recognition, thresholding helps segment licence plate regions from the background by separating foreground objects (such as characters on the licence plate) from the background. Common thresholding techniques include:
 - i. Binary Thresholding: In binary thresholding, pixel values above a certain threshold are set to a maximum value (e.g., 255), while values below the threshold are set to zero. This creates a binary image with clear distinctions between foreground and background regions.
 - ii. Adaptive Thresholding: Adaptive thresholding adjusts the threshold value for different regions of the image based on local intensity variations. This helps in handling images with uneven lighting conditions or varying background intensities more effectively.

7. Edge detection:

- Edge detection algorithms, such as the Canny edge detector, identify edges or boundaries within an image where significant changes in intensity occur. In licence plate recognition, edge detection helps extract important features such as the outlines of licence plate characters. The Canny edge detector, in particular, is widely used for its ability to detect edges accurately while minimising noise and spurious detections.
- Implementation: The Canny edge detection algorithm involves several steps, including smoothing the image with a Gaussian filter to reduce noise, computing the gradient magnitude and direction, applying non-maximum suppression to thin the edges, and finally, applying hysteresis thresholding to detect strong and weak edges.

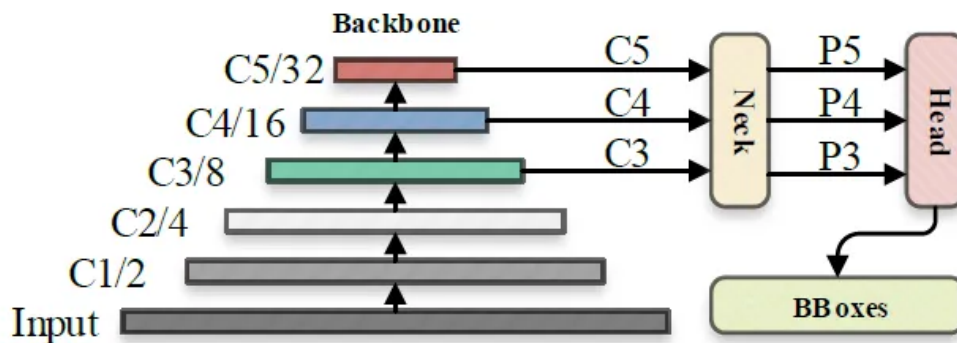
8. Contour Detection:

- Contour detection algorithms, such as OpenCV's `findContours` function, identify continuous curves or boundaries in an image. In licence plate recognition, contour detection helps precisely localise the boundaries of licence plate regions, enabling accurate segmentation and extraction of the licence plate area.
- Implementation: The `findContours` function in OpenCV detects contours by tracing the edges of objects in the binary image obtained from thresholding. It returns a list of contours, each represented as a sequence of points outlining the boundary of a detected object. Contour detection is essential for subsequent processing steps, such as region-of-interest (ROI) selection and character segmentation.

YOLOv8 model architecture

The YOLOv8 model has been gaining significant attention in the field of object detection, and for good reason. Compared to its predecessors, YOLOv8 has demonstrated superior performance in detecting cars and licence plates, making it a highly effective tool for various applications such as traffic monitoring and surveillance. One of the key advantages of YOLOv8 is its ability to detect objects with high precision and speed. This is particularly important in real-time applications where the detection of objects needs to be done quickly and accurately. YOLOv8 has been shown to outperform previous versions in terms of detection speed and accuracy, making it an ideal choice for real-time object detection tasks. Another advantage of YOLOv8 is its ability to detect objects at different scales. This is particularly important in detecting cars and licence plates, which can vary significantly in size. YOLOv8 has been designed to detect objects at different scales, making it highly effective in detecting cars and licence plates of various sizes. Furthermore, YOLOv8 has been optimised for use on different hardware platforms, including CPUs and GPUs. This means that it can be deployed on a wide range of devices, from mobile phones to servers, making it a highly versatile tool for object detection. Our project utilised a pre-trained version of YOLOv8 for detecting the cars and a fine-tuned version of YOLOv8 on a number plate dataset for detecting the number plates on the cars.

High-level architecture



1. Input Processing:

- The input image is processed through an input layer, typically a convolutional neural network (CNN), to extract features. The input layer resizes the image to a fixed size suitable for the network architecture and performs any necessary preprocessing, such as normalisation.

2. Backbone for Feature Extraction:

- The processed image is passed through the backbone network, which is usually a deep CNN architecture like ResNet or Darknet. The backbone network extracts hierarchical features from the input image at multiple spatial scales. These features are represented as feature maps of different sizes, capturing information at different levels of abstraction.

3. Feature Fusion Network (Neck):

- The feature maps obtained from the backbone network are then fused through the feature fusion network, also known as the neck. The purpose of the neck is to combine features from different scales and resolutions to provide a rich representation of the input image. In YOLOv8, the neck typically consists of convolutional layers or feature pyramid networks (FPNs) to aggregate features from different levels of the feature hierarchy.

4. Generation of Feature Maps (P3, P4, P5):

- The fused features are used to generate three feature maps: P3, P4, and P5. These feature maps have spatial dimensions of 80×80 , 40×40 , and 20×20 , respectively, in the case of YOLOv8. These feature maps are resized versions of the original feature maps obtained from the backbone network and are used for detecting objects of different sizes in the input image.

5. Prediction Head:

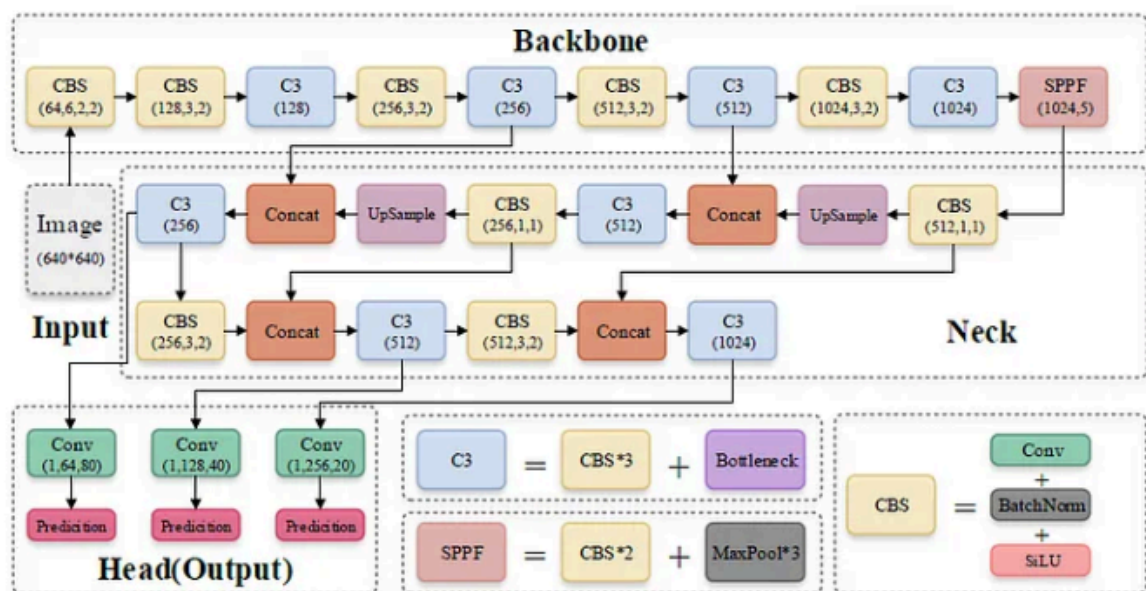
- The feature maps (P3, P4, P5) are then passed through the prediction head, which consists of convolutional layers responsible for predicting object bounding boxes, object classes, and confidence scores. Each pixel in the feature map is associated with a set of anchor boxes, which are predefined bounding boxes of different sizes and aspect ratios. The prediction head

performs confidence calculation and bounding-box regression for each anchor box to predict the presence of objects and their locations in the image.

6. Output Processing:

- The output of the prediction head is a multi-dimensional array (BBboxes) containing object class predictions, class confidence scores, and bounding box coordinates (xmin, ymin, xmax, ymax) for each anchor box. Additional information such as width and height may also be included. The final detection information is obtained by setting confidence and objectness thresholds (confthreshold, objthreshold) to filter out low-confidence predictions and performing a non-maximum suppression (NMS) process to remove redundant detections and generate the final set of detected objects.

Expanded architecture



The above figure shows a detailed architecture of the YOLOv8 model. The model is broken down into three main parts:

1. Backbone: Feature Extraction with CSPDarknet53

- Core:** The backbone of YOLOv8 is a powerful CNN called CSPDarknet53. It's a modified version of Darknet, a popular architecture used in earlier YOLO versions.
- Function:** CSPDarknet53 takes the input image and processes it through a series of convolutional layers. These layers apply filters that extract features from the image, progressively building a richer understanding of the content.
- CSP (Cross Stage Partial Connections):** This is a key innovation in CSPDarknet53. It splits the network into shortcut paths and residual paths. The shortcut paths allow information to flow directly from earlier layers to later ones, preserving valuable details. Residual paths perform additional feature extraction. This combination improves efficiency and accuracy compared to traditional CNNs.
- Bottleneck Layers:** Within CSPDarknet53, bottleneck layers are strategically placed. These layers reduce the dimensionality of feature maps, meaning they decrease the

number of channels in the data. This compression helps to manage the model's size and computational complexity while maintaining informative features.

2. Neck: Feature Fusion with Path Aggregation Network (PAN)

- **Bridge:** The neck stage acts as a bridge between the backbone's feature maps and the head for final detection. In YOLOv8, the PAN (Path Aggregation Network) is employed for this crucial role.
- **FPN (Feature Pyramid Network) Inspiration:** PAN borrows the concept of a Feature Pyramid Network (FPN) used in other detection models. FPN addresses the challenge of detecting objects at various scales within an image.
- **Up-sampling and Concatenation:** PAN takes feature maps from different stages of the backbone, representing different resolutions. It up-samples the lower-resolution maps to match the size of higher-resolution ones. Then, it concatenates these feature maps, creating a richer representation that combines coarse (global) and fine-grained (local) information. This is crucial for accurate detection of objects of varying sizes.
- **PAN's Innovation:** PAN goes a step further than FPN by introducing bottom-up path aggregations. It adds lower-level features directly from the backbone to the upsampled pathways. This injects more precise details about object locations, enhancing the model's ability to localise objects accurately.

3. Head: Detection with YOLO Layer

- **Prediction:** The head, represented by the YOLO layer, takes the processed feature maps from the neck as input and performs the final detection task.
- **Bounding Boxes and Class Probabilities:** The YOLO layer outputs a set of bounding boxes and their corresponding confidence scores. A bounding box is a rectangular region that encloses a detected object in the image. The confidence score, represented as a probability, indicates how certain the model is that the bounding box contains a specific object class.
- **Anchor Boxes and Class Prediction:** During training, the model learns a set of pre-defined anchor boxes with different sizes and aspect ratios. These anchor boxes serve as a reference for predicting bounding boxes for various objects. The YOLO layer also predicts the class probabilities for each bounding box, indicating which object category (e.g., car, person, dog) the model believes is present within the box.

Deep sort model architecture

DeepSORT combines the power of deep learning-based object detection with a sophisticated tracking algorithm to achieve highly accurate and robust object tracking in real-time scenarios. Unlike traditional tracking methods that rely solely on motion estimation or appearance-based features, DeepSORT leverages deep neural networks for both detection and tracking, resulting in superior performance across a wide range of applications. At its core,

DeepSORT extends the SORT (Simple Online and real-time tracking) algorithm by integrating deep association embeddings into the tracking process. This integration enables DeepSORT to not only track objects reliably over time but also effectively handle challenges such as occlusions, appearance variations, and object re-identification in complex scenes. Following are the key components of the DeepSORT algorithm:

1. Object Detection:

- Deep SORT typically begins with an object detection step, which involves using a pre-trained deep learning model such as YOLO (You Only Look Once), Faster R-CNN, or SSD (Single Shot Multibox Detector). This model analyzes each frame of a video or an image sequence and identifies objects within the scene. It provides bounding boxes around these objects along with corresponding confidence scores indicating the model's certainty about the detections.

2. Feature Extraction:

- Once objects are detected, Deep SORT extracts features from these objects. Features capture various visual characteristics such as color, texture, and shape, which are crucial for distinguishing between different objects. Deep SORT typically employs a deep neural network to extract high-level features that represent the appearance of each detected object effectively.

3. Data Association:

- The core of Deep SORT lies in its ability to associate objects across frames, especially in scenarios where objects may appear, disappear, or undergo occlusion. Deep SORT predicts the likely positions of objects in the current frame based on their positions in the previous frame and matches these predictions with the newly detected objects. This process involves:
 - Motion Prediction: Deep SORT predicts where objects are likely to be in the current frame based on their velocities and accelerations from previous frames.
 - Appearance Matching: Alongside motion prediction, Deep SORT compares the appearance features of detected objects to previously tracked objects. It uses techniques like the Hungarian algorithm or the Kalman filter to optimize the assignment of detections to existing tracks, considering both appearance similarities and motion predictions.

4. Tracking:

- With associations established, Deep SORT maintains tracks for each object over time. It updates the state of each track based on new detections and associations. The state typically includes information such as position, velocity, acceleration, and a unique identifier for the object. Deep SORT also incorporates mechanisms to handle occlusions, and robustly track interruptions and appearance changes. For example, when an object is occluded by another object, Deep SORT predicts its trajectory until it becomes visible again.

5. Output:

- Finally, Deep SORT provides the tracked objects' trajectories and identities as output. Each tracked object is assigned a unique identifier, allowing for consistent tracking across frames. Deep SORT may also provide additional information such as confidence scores, track lengths, and object attributes depending on the application requirements.

OCR using EasyOCR

Optical Character Recognition (OCR) is a technology that enables computers to recognize text within digital images or scanned documents, converting it into machine-readable text. For extracting the text from number plates, we used EasyOCR. It is a Python library that simplifies OCR tasks by providing an easy-to-use interface for performing text recognition on images. EasyOCR is built on top of the PyTorch deep learning framework and utilises pre-trained models to recognize text in various languages and scripts. It offers a straightforward API for performing OCR tasks with different configurations.

Typical Workflow with EasyOCR:

1. Installation: EasyOCR can be installed via pip, and it has minimal dependencies, making it easy to set up in various environments.
2. Image Input: Users provide input images or documents containing text that they want to recognize.
3. Text Recognition: EasyOCR processes the input images and detects text regions using its pre-trained models. It then recognizes the text within these regions and outputs the results in a structured format.
4. Output Processing: The recognized text can be further processed or utilised according to the application's needs. This may include tasks such as text extraction, translation, or integration with other systems.

Use Cases of EasyOCR:

- Document Digitization: EasyOCR can be used to convert scanned documents, PDFs, or images of printed text into editable and searchable digital formats.
- Text Extraction from Images: It enables extracting text from images captured by cameras, smartphones, or other devices, facilitating tasks like data entry, form processing, and information retrieval.
- Automated Text Analysis: EasyOCR can assist in automating tasks that require analysing and understanding text content, such as sentiment analysis, keyword extraction, and content summarization.

Graphical user interface

Tkinter is a standard GUI (Graphical User Interface) toolkit for Python. It provides a set of tools for building desktop applications with graphical elements such as windows, buttons,

labels, and entry fields. The interface of the application was built using Tkinter “frames”. Frames in Tkinter refer to container widgets that are used to organise and group other widgets together within a window or application. Frames are often used to create a structured layout for organising the graphical elements of a GUI. In our application, we had 3 frames:

1. Top Frame:
 - The top frame is created using `tk.Frame(master)`.
 - It contains the label prompting the user to enter authorised number plates and an entry widget where the user can input the number plates.
 - This frame is responsible for organising and displaying these widgets in a specific layout within the GUI.
2. Middle Frame:
 - The middle frame is also created using `tk.Frame(master)`.
 - It contains the "Process Video" button and the video label.
 - The "Process Video" button triggers the video processing task when clicked, while the video label displays the processed video frames during playback.
 - Similar to the top frame, the middle frame helps organise and display these widgets in a structured manner within the GUI.
3. Bottom Frame:
 - The bottom frame, again created using `tk.Frame(master)`, holds the "Play" and "Pause" buttons.
 - These buttons allow the user to control the playback of the processed video.
 - Like the other frames, the bottom frame facilitates the layout and organisation of these control buttons within the GUI.

Following are the features of the interface:

1. Video Processing:
 - When the user clicks the "Process Video" button, the `process_video` method is called. This method extracts the authorised number plates entered by the user, processes the video, and displays an appropriate message dialog indicating the success or failure of the video processing task.
2. Video Playback:
 - The "Play" button triggers the `play_video` method, which initialises video playback by capturing frames from the processed video and displaying them in the video label.
 - The "Pause" button, when clicked, pauses the video playback by setting the `video_playing` variable to `False`.
3. Video Stopping:
 - The `stop` method is responsible for releasing the video capture object and resetting the `video_playing` variable to `False` when the video playback is stopped or completed.
4. Input licence plates:

- This is a text box, where the user can enter the licence plates, separated by a comma. This list of plates act as the authorised vehicles and the model will process the video based on this list. If any plate is detected that is not present in the list, the model will highlight the vehicle as unauthorised.

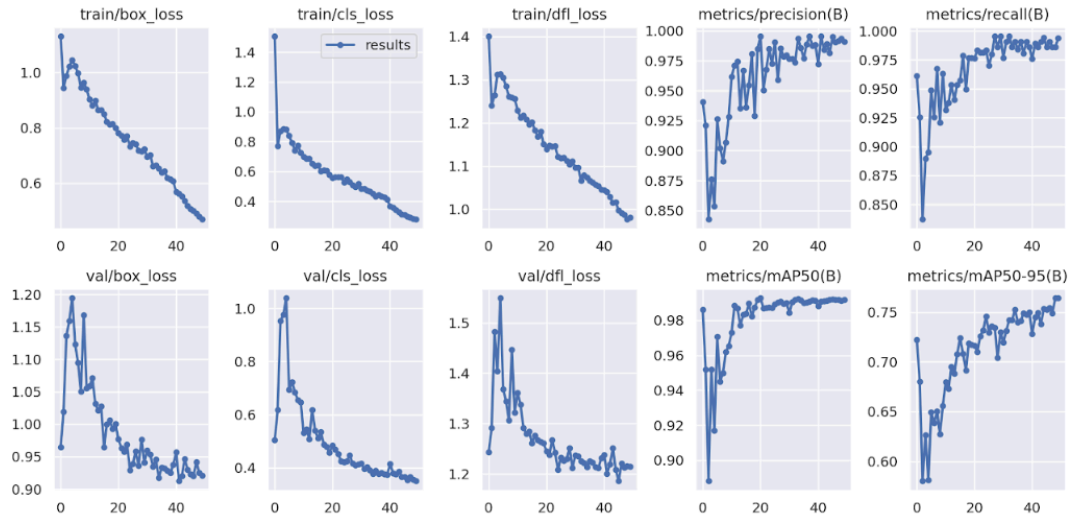
Results and evaluations

The key aspect of our system is the training of the YOLOv8 model on the licence plate dataset. The YOLOv8 model fine-tuned on the licence plate dataset provided optimal results. Below we will discuss the performance metrics and other evaluations of the YOLOv8 model on the licence plate dataset. The following metrics were used to assess the quality of the trained YOLOv8 model:

1. **train/box_loss**: This represents the loss associated with the bounding box predictions during training. In YOLO (You Only Look Once) models, bounding box prediction is a crucial aspect, and this loss measures the discrepancy between the predicted bounding boxes and the ground truth bounding boxes.
2. **train/cls_loss**: This is the loss associated with the class predictions during training. In YOLO models, each bounding box predicts the objectness score (confidence that an object is present) and the class probabilities. This loss measures the accuracy of class predictions.
3. **train/dfl_loss**: DFL is "directional feature learning." This loss is specific to YOLOv8 and is related to the feature learning process for detecting oriented objects such as license plates. It helps the model learn features that are useful for accurately detecting objects with specific orientations.
4. **metrics/precision(B)**: Precision is a metric that measures the proportion of true positive predictions (correctly identified licence plates) out of all the positive predictions made by the model.
5. **metrics/recall(B)**: Recall is a metric that measures the proportion of true positive predictions out of all actual positive instances (licence plates) in the dataset. The "(B)" may refer to recall calculated specifically for bounding boxes.
6. **metrics/mAP50(B)**: mAP (mean Average Precision) at 50% overlap threshold for bounding boxes. It's a popular metric for evaluating object detection models, considering both precision and recall across different object categories. mAP50 calculates the average precision when considering a predicted bounding box as correct if it has an overlap of at least 50% with a ground truth bounding box.
7. **metrics/mAP50-95(B)**: This is similar to mAP50 but calculates the mean Average Precision across a range of overlap thresholds from 50% to 95%. This provides a more comprehensive evaluation of the model's performance across different levels of bounding box overlap.
8. **val/box_loss, val/cls_loss, val/dfl_loss**: These are similar to their training counterparts but are calculated on a separate validation dataset. They indicate how well the model generalises to unseen data by measuring the loss associated with

bounding box predictions, class predictions, and directional feature learning on the validation set.

The YOLOv8 model was trained for 50 epochs, taking around 2 hours of training and testing time combined, on a Google Colab notebook, utilising its free GPU capabilities.

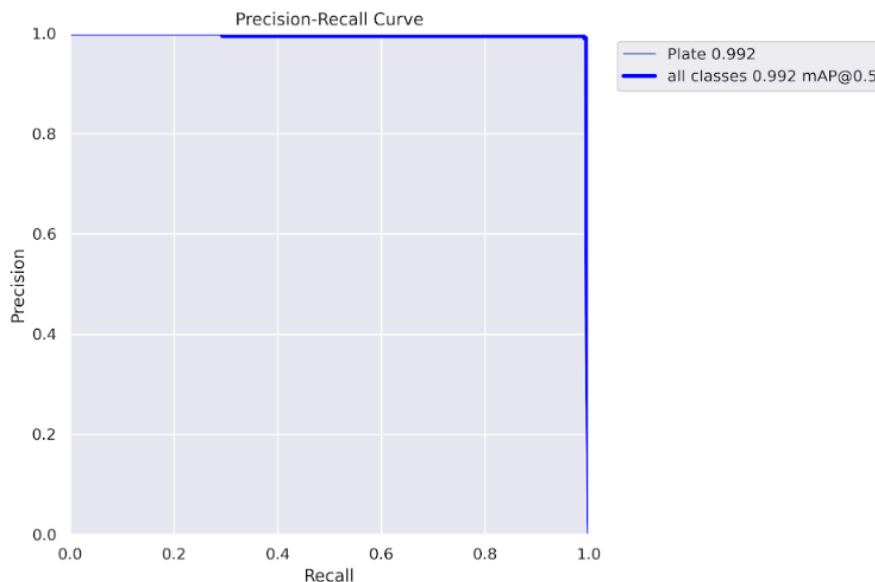


The picture above shows how the losses decreased and the precision metrics increased throughout the 50 epochs for the training process. We can note that there were a few fluctuations in the training process as evident from the figure but this is a normal part of the process.

epoch	train/box_loss	train/cls_loss	train/dfl_loss	metrics/precision(B)	metrics/recall(B)	metrics/mAP50(B)	metrics/mAP50-95(B)	val/box_loss	val/cls_loss	val/dfl_loss
45	0.50679	0.30989	0.99735	0.99498	0.98605	0.99214	0.7526	0.92286	0.36587	1.1856
46	0.50029	0.29936	0.99169	0.99028	0.9907	0.99182	0.75454	0.91998	0.35376	1.2206
47	0.4907	0.29422	0.98731	0.99168	0.98605	0.99192	0.74906	0.94152	0.36653	1.2122
48	0.47925	0.28516	0.97676	0.99347	0.98605	0.99144	0.76426	0.92527	0.35635	1.2154
49	0.46986	0.28009	0.98088	0.99073	0.99376	0.99173	0.76432	0.92106	0.35051	1.2146

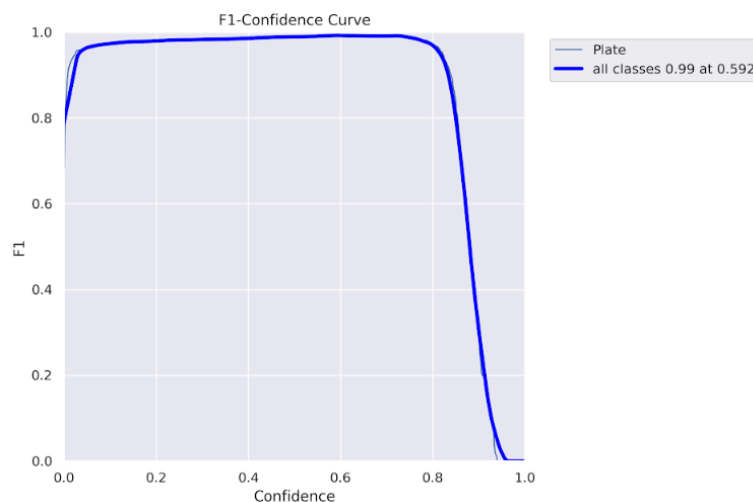
The table above shows the training metrics of the YOLOv8 model over the course of the last 5 epochs. Overall, there's a positive trend observed in the model's training, with decreasing losses indicating improved precision in bounding box prediction, object classification, and feature refinement. Notably, precision and recall metrics for bounding boxes remain consistently high, suggesting effective object identification with minimal false positives and

negatives. Moreover, the mean Average Precision (mAP) scores demonstrate an overall improvement or stability, affirming the model's proficiency in object detection tasks. Thus, the model demonstrates promising performance, but continuous monitoring and adjustments are essential to ensure sustained generalisation and prevent overfitting issues.



Based on the precision-recall curve in the above image, the model seems to have good performance. Here are some key takeaways from the graph:

- The curve reaches a high value (almost 1.0) for both precision and recall. This means that the model can correctly identify most of the number plates (high precision) and it's also able to find most of the number plates in the image (high recall).
- The mAP@0.5 value of 0.992 is also very good, which indicates high average precision across different Intersections over Union (IoU) thresholds. IoU is a metric used in object detection to measure how well a detection box overlaps with the ground truth box.



Here are some key takeaways from the graph:

- The confidence level is a measure of how certain the model is that a particular prediction is correct. In this graph, the model is very confident (confidence level of 0.592) when it achieves an F1 score of 0.99.
- The F1 score takes into account both precision and recall. Precision is a measure of how many of the positive predictions were actually correct, and recall is a measure of how many of the actual positive cases were identified correctly by the model.

Based on above considerations, we conclude that the YOLOv8 model fine-tuned on the licence plate dataset performed considerably well for a total of 50 epochs with optimal memory usage and adequate performance. Further areas of improvement could include increasing the dataset size to account for difference in lighting conditions in the images, and training the model for a higher number of epochs on a high-end GPU-based environment.

Challenges and limitations

In this report, we discussed the effectiveness of our object detection system, primarily built around the YOLOv8 model, in accurately identifying cars and licence plates in various environments. The utilization of the YOLOv8 architecture demonstrated promising results, showcasing its potential in real-world applications. Despite its capabilities, our journey was not without hurdles. One notable challenge was encountered during the training phase, where memory constraints posed significant limitations, hindering the full exploration of the YOLOv8 model's capabilities. This limitation led to a constrained training process and potentially impacted the model's ultimate performance. Moreover, the dataset used for fine-tuning the YOLOv8 model exhibited limitations, particularly in terms of size and diversity. The dataset's restricted scope could have potentially hindered the model's ability to generalise effectively across different scenarios and conditions.

In response to these challenges, extensive preprocessing techniques were implemented to enhance the dataset's quality and diversity. Augmentation methods such as brightness adjustments, contrast enhancements, and data augmentation were applied to augment the dataset, enriching it with variations and bolstering the model's robustness. Additionally, the training process was meticulously optimised to accommodate the memory constraints, allowing for efficient utilisation of available resources. Despite these challenges, the YOLOv8 model exhibited promising performance in detecting cars and licence plates, showcasing its effectiveness in real-world scenarios.

Moving forward, several avenues for improvement could be explored to further enhance the system's performance. Firstly, efforts should be directed towards expanding the dataset by collecting more diverse and representative samples. Increasing the dataset's size and diversity would enable the model to generalise more effectively across different environments and conditions. Furthermore, exploring alternative model architectures or optimization techniques to mitigate memory constraints could unlock the full potential of more advanced models such as YOLOv8, potentially leading to even higher detection accuracy and efficiency.

Conclusion

Throughout the project journey, various methods for automatic number plate recognition (ANPR) were explored and reviewed. The project proposed a novel method for ANPR, leveraging different state-of-the-art techniques. The result was an optimal and performant model, suitable for integration into hardware devices for real-time number plate detection and authorization.

The project didn't stop at merely reviewing existing techniques. It went further to propose a new approach, combining different state-of-the-art techniques to develop a model that not only performs optimally but is also efficient.

The project ensured practicality by designing the method to seamlessly integrate with hardware devices. This enables the system to detect and authorise number plates in real-time, making it suitable for use in real-world scenarios.

References

Omar, N., Al-Ali, S. G., & Sengur, A. (2019). An Efficient Model for Automatic Number Plate Detection using HOG Feature from New North Iraq Vehicle Images Dataset

Li, H., Wang, P., & Shen, C. (2019). Toward end-to-end car licence plate detection and recognition with deep neural networks. *IEEE Transactions on Intelligent Transportation Systems*, 20(3), 1126-1136. <https://doi.org/10.1109/tits.2018.2847291>

Luo, S. and Liu, J. (2022). Research on car licence plate recognition based on improved yolov5m and lprnet. *IEEE Access*, 10, 93692-93700. <https://doi.org/10.1109/access.2022.3203388>

Vyvere, B. and Colpaert, P. (2022). Using anpr data to create an anonymized linked open dataset on urban bustle. *European Transport Research Review*, 14(1). <https://doi.org/10.1186/s12544-022-00538-1>

Narendra, A., Choudhury, S., Mishra, A., & Misra, I. (2023). Chaurah: a smart Raspberry pi based parking system.. <https://doi.org/10.36227/techrxiv.21931398>

Zhu, Q., Liu, Y., Zhao, Z., & Xin, M. (2022). Research on licence plate location algorithm based on yolov5. *Journal of Physics Conference Series*, 2278(1), 012040. <https://doi.org/10.1088/1742-6596/2278/1/012040>

Shah, N. and Shah, M. (2021). Crime forecasting: a machine learning and computer vision approach to crime prediction and prevention. *Visual Computing for Industry Biomedicine and Art*, 4(1). <https://doi.org/10.1186/s42492-021-00075-z>

Barroso, J., Oliveira, J., & Oliveira-Neto, F. (2021). The use of traffic data from automatic monitoring systems to obtain day-to-day time series of vehicle traffic volumes and origin-destination flows in urban networks. *Transportes*, 29(2). <https://doi.org/10.14295/transportes.v29i2.2385>

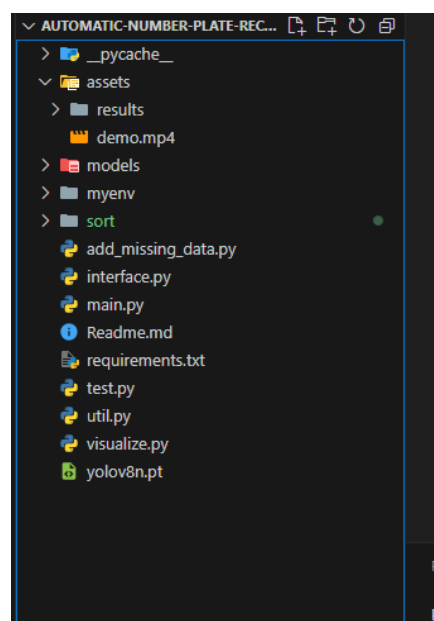
Sánchez-Cambronero, S., Álvarez-Bazo, F., Rivas, A., & Gallego, I. (2021). Dynamic route flow estimation in road networks using data from automatic number of plate recognition sensors. *Sustainability*, 13(8), 4430. <https://doi.org/10.3390/su13084430>

Mane, P. B., Bade, G., & Patil, V. (2019). Image processing based toll automation technique using anpr. *Helix*, 9(3), 4926-4930. <https://doi.org/10.29042/2019-4926-4930>

Rehman, S. U., Ahmad, M., Nawaz, A., & Ali, T. (2020). An efficient approach for vehicle number plate recognition in pakistan. *The Open Artificial Intelligence Journal*, 06(1), 12-21. <https://doi.org/10.2174/1874061802006010012>

Alizadeh, F. and Luqman, S. (2021). A novel approach for automatic car plate detection and recognition. *UKH Journal of Science and Engineering*, 5(2), 1-9. <https://doi.org/10.25079/ukhjse.v5n2y2021.pp1-9>

Appendix



1. From the above figure, we can see the file structure. In it, the assets folder contains the “Demo.mp4” video file as the input video. While the subfolder “results” contains the results generated by the program.

- The “models” folder contains the fine-tuned YOLOv8 model’s weights.
- The “sort” folder contains the source code for the DeepSORT algorithm, taken from [this](#) link.
- The “myenv” folder is the virtual environment.
- The main file is the “main.py” file which takes the input video from the assets folder and generates a CSV file named “test.csv” containing the YOLOv8 model’s detections and the extracted number plates, and is saved in the “assets/results/test.csv” location. The figure below shows the contents of the “test.csv” file:

A	B	C	D	E	F	G	H
frame_nr	car_id	car_bbox	license_plate_bbox	license_plate_bbox	license_plate_bbox	license_number	license_score
13	1	[279.4689306144358 362.7154848030421 489.432800651155 526.34075032]	[352.71630859375 469.2749338964844 404.64007568359375 487.4782405]			0.680534981	0.128398684
20	1	[271.7679714443269 375.17760232708616 484.5632058301732 540.4078279]	[346.58837890625 483.3661193847656 400.6454162597656 500.47742335]			0.734623611	0.260230398
24	1	[265.5124432855597 383.1968751907514 484.372089070376 551.73443355]	[343.3096618652344 491.2461853027344 400.9673156738281509.3790588]			0.68758241	0.070372863
29	1	[262.6927722117063 391.7338628816633 485.9656337725177 563.88190852]	[335.03375244140625 503.0223388671875 403.66046142578125 522.64562]			0.371027023	0.033408511
35	1	[252.84694101396198 403.40045903438195 481.7041934432265 579.063844]	[334.83746337890625 516.98681640625 396.58709716796875 536.097900]			0.789984524	0.194250396
36	1	[252.193486664818 405.23724500356974 481.6941830683631 581.5934238]	[333.07342523296875 521.0964965820312 393.68505859375 536.6223754]			0.698814392	0.212001982
37	1	[251.75404246588095 406.9698670959906 481.74377332588076 583.269306]	[285.7931213378906 526.5478515625 397.1750793457031 543.3858642578]			0.265112251	0.485253329
41	1	[247.26001953957884 414.4498002657427 479.3323233951479 587.337725]	[327.3353881835375 533.4781870117188 367.05377197265625 550.181274]			0.744374613	0.119022532
44	3	[598.2767890346577 306.344588762579 759.1155533395479 446.24819276]	[656.7556762695312 401.3562327246094 712.677001953125 415.32032895]			0.739592254	0.106812539
50	6	[655.5535501918506 438.9587287710348 871.0221585012887 593.345151862]	[738.4633178710938 553.4417724609375 803.8519287109375 573.8088389]			0.533144355	0.201803056
52	3	[598.013091813897 316.6693527060931 763.1137942392613 458.902647364]	[659.045166015625 410.73583984375 714.581848145312 426.35604858396]			0.825406432	0.523334149
55	6	[658.7533315191074 451.34453238179736 879.277318938746 598.33271722]	[739.173583984375 567.218200683538 804.5213012695312 588.83538818]			0.664381325	0.075748393
56	3	[597.8407548347586 320.96642267481377 766.1404422198432 464.2385521]	[658.773868164062 415.4673767089844 718.1727905273438 430.6634335]			0.801673949	0.505591507
57	3	[597.5415135482737 321.12438526583522 766.5359036121205 464.66272897]	[658.896181640625 415.4830322265625 718.4002075195312 430.68011474]			0.802024364	0.505591507
59	6	[658.2792751068745 459.8146617443716 882.0167442136195 598.84600854]	[746.373291015625 581.8545532226562 807.7193603515625 596.0]			0.681887595	0.073886334
60	3	[599.0195641526364 324.2623868505539 763.7333637069877 468.7551986]	[658.005796339438 418.3038024902344 721.4282636914062 435.223907]			0.802604735	0.194682526
63	3	[597.7930919053552 328.97890424013494 771.9780323640552 475.0296321]	[662.74072265625 425.1299143652344 719.0070190429688 440.428161621]			0.822451075	0.079521673
65	3	[598.1737898951496 331.8076822167243 772.5726150425642 478.948571215]	[661.9230346679688 426.8167724809375 724.9071044321875 443.08615112]			0.859328012	0.116206773
66	3	[598.3108110731011 332.961507908414 773.3186639083991 480.49745531982]	[661.92578125 428.4246826717675 723.5867919321875 445.320806884765]			0.830386281	0.267162889
67	3	[598.266015483918 333.10639682413787 773.6739566411645 481.133505834]	[661.9581909179688 428.42633056640625 723.6426391601562 445.320501]			0.830801368	0.317648552
68	3	[598.9670702358835 334.4408835822664 774.8317103988463 483.3927848]	[662.045166015625 430.185332519531 724.57080078125 447.9089660644]			0.846855342	0.121303671
70	3	[599.1575191460057 337.0313582751106 776.1075915605593 487.070205845]	[662.739013671875 434.75897216796875 726.47265625 449.97448730468]			0.751244247	0.185509396
71	3	[599.061169253025 337.797596123268 776.2817970359486 488.400454596]	[665.482177734375 436.2407537138281 726.3878784179688 451.132871093]			0.78145206	0.176507251

- The second file we need to run is the “add_missing_data.py” to format and interpolate the "test.csv" file. This script will create a new file called "test_interpolated.csv" file stored in the assets/results folder as well.
- The third file to run is the “visualize.py” file. This will take the "test_interpolated.csv" file and the input video (stored in "assets/demo.mp4") and overlay the results of the "test_interpolated.csv" file onto the video. It will store the resultant video in the "assets/results/out.mp4" location.
- The “utils.py” file contains utility and helper functions for running the other modules of the code.
- Lastly, we can run the “interface.py” file to run the interface. It will ask for a list of license plates as inputs (for example: ATVET342,VETV25,VTGV35), and then you have to submit the list. After a few seconds, you will get the output video in the interface with full detections, highlighting authorized plates in "GREEN" and unauthorized plates in "RED".
- The “requirements.txt” file contains the libraries for installing everything.